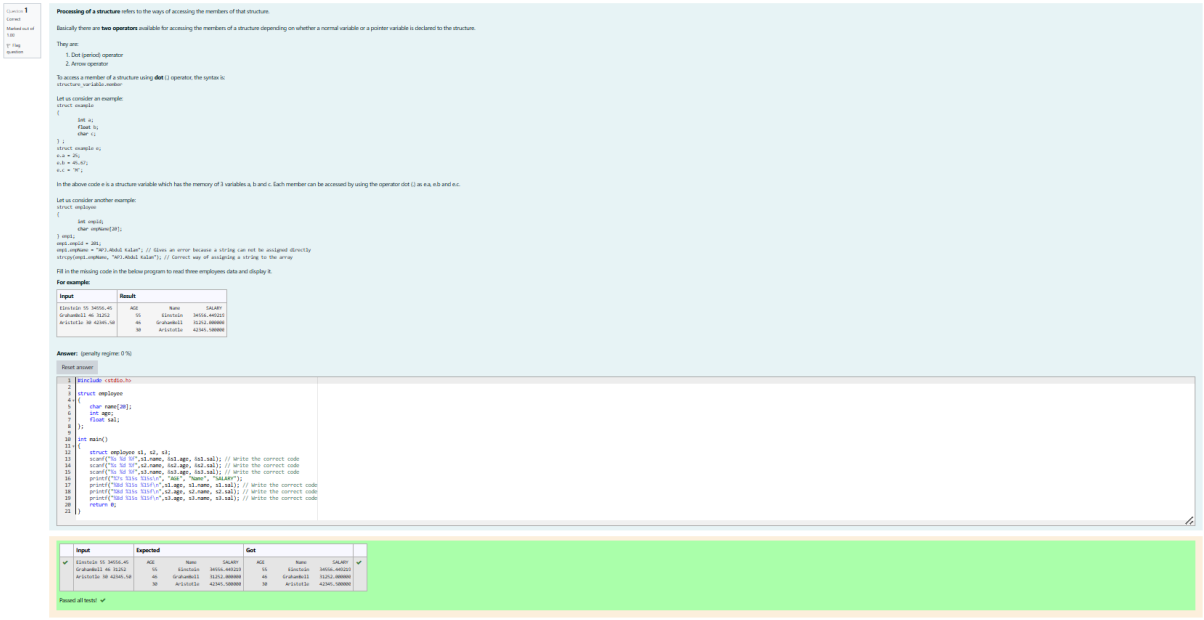
WEEK 5



Processing of a structure refers to the ways of accessing the members of that structure.

Basically there are **two operators** available for accessing the members of a structure depending on whether a normal variable or a pointer variable is declared to the structure.

They are:

1. Dot (period) operator
2. Arrow operator

To access a member of a structure using **dot** (.) operator, the syntax is:

```
structure_variable.member
```

Let us consider an example:

```
struct example
{
        int a;
        float b;
        char c;
} ;
struct example e;
e.a = 25;
e.b = 45.67;
e.c = 'M';
```

In the above code e is a structure variable which has the memory of 3 variables a, b and c. Each member can be accessed by using the operator dot (.) as e.a, e.b and e.c.

Let us consider another example:

```
struct employee
{
        int empId;
        char empName[20];
} emp1;
emp1.empId = 201;
emp1.empName = "Md.Abdul Kalam"; // Gives an error because a string can not be assigned directly
strcpy(emp1.empName, "Md.Abdul Kalam"); // Correct way of assigning a string to the array
```

Fill in the missing code in the below program to read three employees data and display it.

For example:

Input	Result
Einstein 55 34556.45 Graham11 46 31252 Aristotle 50 42345.50	AGE Name SALARY 55 Einstein 34556.445223 46 Graham11 31252.000000 50 Aristotle 42345.500000

Answer: (penalty regime: 0 %)

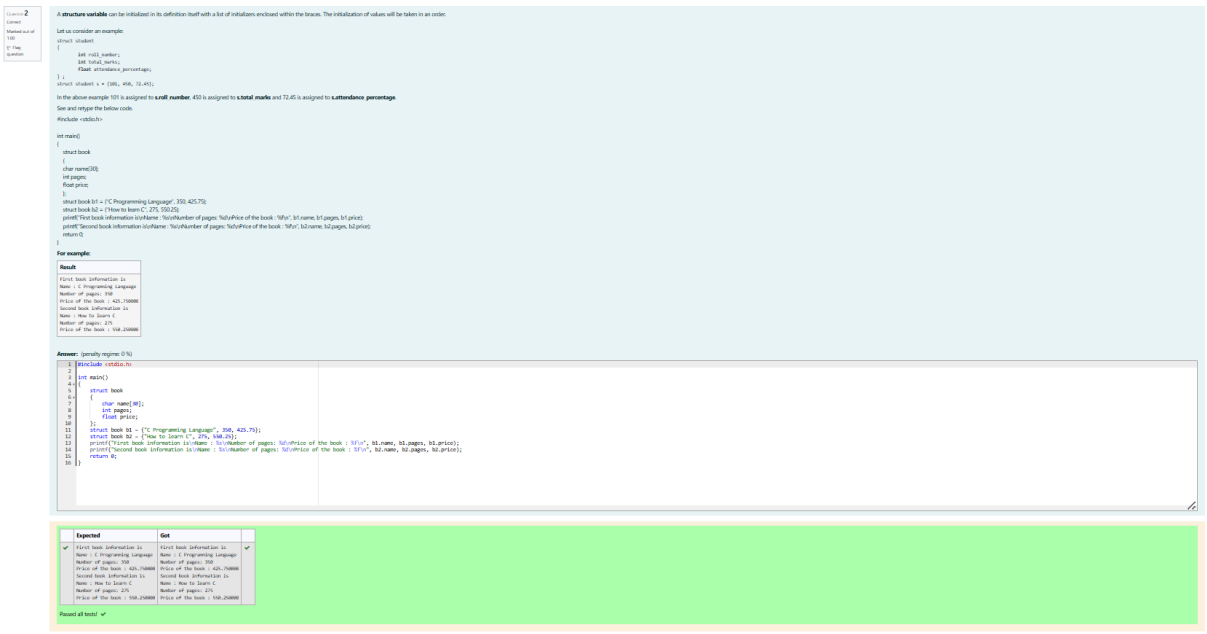
Reset answer

```c
#include <stdio.h>

struct employee
{
    char name[20];
    int age;
    float sal;
};

int main()
{
    struct employee s1, s2, s3;
    scanf("%s %d %f",s1.name, &s1.age, &s1.sal); // Write the correct code
    scanf("%s %d %f",s2.name, &s2.age, &s2.sal); // Write the correct code
    scanf("%s %d %f",s3.name, &s3.age, &s3.sal); // Write the correct code
    printf("%7s %3s %10s\n", "AGE", "Name", "SALARY");
    printf("%3d %3s %3f\n",s1.age, s1.name, s1.sal); // Write the correct code
    printf("%3d %3s %3f\n",s2.age, s2.name, s2.sal); // Write the correct code
    printf("%3d %3s %3f\n",s3.age, s3.name, s3.sal); // Write the correct code
    return 0;
}
```

	Input	Expected	Got
✔	Einstein 55 34556.45 Graham11 46 31252 Aristotle 50 42345.50	AGE Name SALARY 55 Einstein 34556.445223 46 Graham11 31252.000000 50 Aristotle 42345.500000	AGE Name SALARY 55 Einstein 34556.445223 46 Graham11 31252.000000 50 Aristotle 42345.500000

Passed all tests! ✔

➔Thus, program is executed successfully.



A **structure variable** can be initialized in its definition itself with a list of initializers enclosed within the braces. The initialization of values will be taken in an order.

Let us consider an example:

```
struct student
{
        int roll_number;
        int total_marks;
        float attendance_percentage;
} ;
struct student s = {101, 450, 72.45};
```

In the above example 101 is assigned to **s.roll_number**, 450 is assigned to **s.total marks** and 72.45 is assigned to **s.attendance percentage**

See and retype the below code.

```
#include <stdio.h>

int main()
{
   struct book
   {
   char name[30];
   int pages;
   float price;
   };
   struct book b1 = {"C Programming Language", 350, 425.75};
   struct book b2 = {"How to learn C", 275, 550.25};
   printf("First book information is\nName : %s\nNumber of pages: %d\nPrice of the book : %f\n", b1.name, b1.pages, b1.price);
   printf("Second book information is\nName : %s\nNumber of pages: %d\nPrice of the book : %f\n", b2.name, b2.pages, b2.price);
   return 0;
}
```

For example:

Result
First book information is Name : C Programming Language Number of pages: 350 Price of the book : 425.750000 Second book information is Name : How to learn C Number of pages: 275 Price of the book : 550.250000

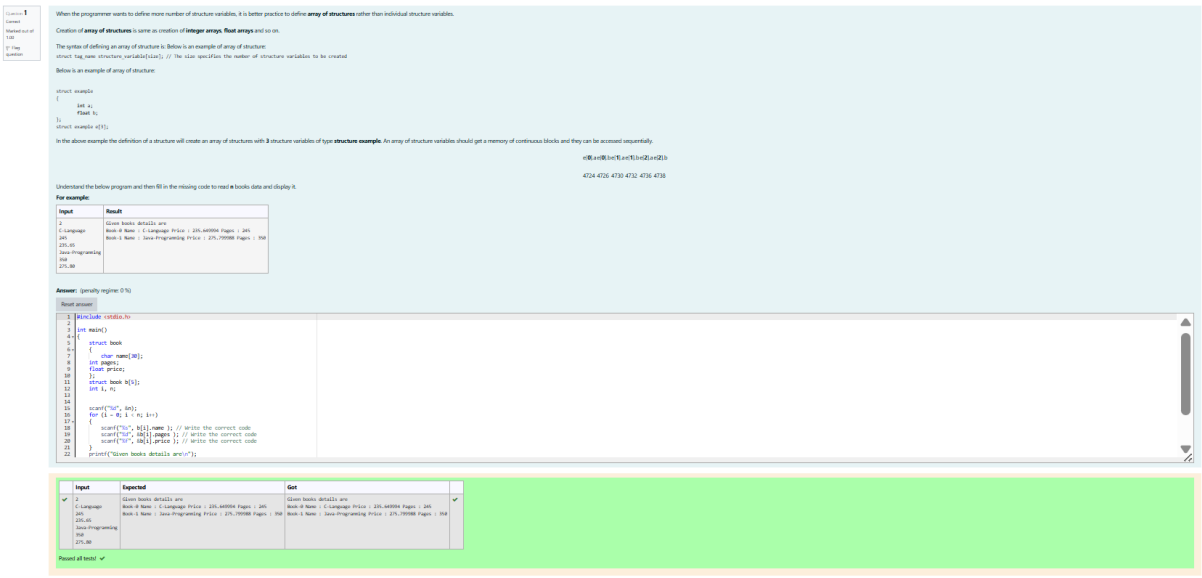
Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int main()
{
    struct book
    {
        char name[30];
        int pages;
        float price;
    };
    struct book b1 = {"C Programming Language", 350, 425.75};
    struct book b2 = {"How to learn C", 275, 550.25};
    printf("First book information is\nName : %s\nNumber of pages: %d\nPrice of the book : %f\n", b1.name, b1.pages, b1.price);
    printf("Second book information is\nName : %s\nNumber of pages: %d\nPrice of the book : %f\n", b2.name, b2.pages, b2.price);
    return 0;
}
```

	Expected	Got
✔	First book information is Name : C Programming Language Number of pages: 350 Price of the book : 425.750000 Second book information is Name : How to learn C Number of pages: 275 Price of the book : 550.250000	First book information is Name : C Programming Language Number of pages: 350 Price of the book : 425.750000 Second book information is Name : How to learn C Number of pages: 275 Price of the book : 550.250000

Passed all tests! ✔

➔Thus, program is executed successfully.



When the programmer wants to define more number of structure variables, it is better practice to define **array of structures** rather than individual structure variables.

Creation of **array of structures** is same as creation of **integer arrays**, **float arrays** and so on.

The syntax of defining an array of structure is: Below is an example of array of structure:

```
struct tag_name structure_variable[size]; // The size specifies the number of structure variables to be created
```

Below is an example of array of structure:

```
struct example
{
    int a;
    float b;
};
struct example a[3];
```

In the above example the definition of a structure will create an array of structures with **3** structure variables of type **structure example**. An array of structure variables should get a memory of continuous blocks and they can be accessed sequentially.

a[0].a a[0].b a[1].a a[1].b a[2].a a[2].b

4724 4726 4730 4732 4736 4738

Understand the below program and then fill in the missing code to read **n** books data and display it.

For example:

Input	Result
2 C-Language 245 235.65 Java-Programming 350 275.80	Given books details are Book-0 Name : C-Language Price : 235.649994 Pages : 245 Book-1 Name : Java-Programming Price : 275.799988 Pages : 350

Answer: (penalty regime: 0 %)

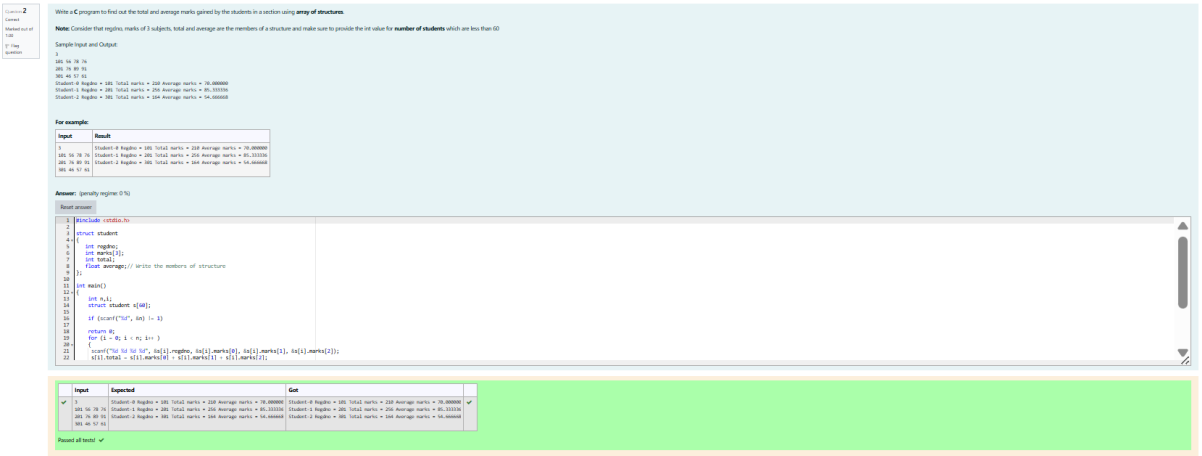
Reset answer

```
#include <stdio.h>

int main()
{
    struct book
    {
        char name[30];
    int pages;
    float price;
    };
    struct book b[5];
    int i, n;

    scanf("%d", &n);
    for (i = 0; i < n; i++)
    {
        scanf("%s", b[i].name ); // Write the correct code
        scanf("%d", &b[i].pages ); // Write the correct code
        scanf("%f", &b[i].price ); // Write the correct code
    }
    printf("Given books details are\n");
```

	Input	Expected	Got
✔	2 C-Language 245 235.65 Java-Programming 350 275.80	Given books details are Book-0 Name : C-Language Price : 235.649994 Pages : 245 Book-1 Name : Java-Programming Price : 275.799988 Pages : 350	Given books details are Book-0 Name : C-Language Price : 235.649994 Pages : 245 Book-1 Name : Java-Programming Price : 275.799988 Pages : 350

Passed all tests! ✔

➔Thus, program is executed successfully.



Write a C program to find out the total and average marks gained by the students in a section using **array of structures**.

Note: Consider that regdno, marks of 3 subjects, total and average are the members of a structure and make sure to provide the int value for **number of students** which are less than 60

Sample Input and Output:

```
3
101 56 78 76
201 76 89 91
301 46 57 61
Student-0 Regdno = 101 Total marks = 210 Average marks = 70.000000
Student-1 Regdno = 201 Total marks = 256 Average marks = 85.333336
Student-2 Regdno = 301 Total marks = 164 Average marks = 54.666668
```

For example:

Input	Result
3 101 56 78 76 201 76 89 91 301 46 57 61	Student-0 Regdno = 101 Total marks = 210 Average marks = 70.000000 Student-1 Regdno = 201 Total marks = 256 Average marks = 85.333336 Student-2 Regdno = 301 Total marks = 164 Average marks = 54.666668

Answer: (penalty regime: 0 %)

Reset answer

```
#include <stdio.h>

struct student
{
    int regdno;
    int marks[3];
    int total;
    float average;// Write the members of structure
};

int main()
{
    int n,i;
    struct student s[60];

    if (scanf("%d", &n) != 1)

    return 0;
    for (i = 0; i < n; i++ )
    {
        scanf("%d %d %d %d", &s[i].regdno, &s[i].marks[0], &s[i].marks[1], &s[i].marks[2]);
        s[i].total = s[i].marks[0] + s[i].marks[1] + s[i].marks[2];
```

	Input	Expected	Got
✔	3 101 56 78 76 201 76 89 91 301 46 57 61	Student-0 Regdno = 101 Total marks = 210 Average marks = 70.000000 Student-1 Regdno = 201 Total marks = 256 Average marks = 85.333336 Student-2 Regdno = 301 Total marks = 164 Average marks = 54.666668	Student-0 Regdno = 101 Total marks = 210 Average marks = 70.000000 Student-1 Regdno = 201 Total marks = 256 Average marks = 85.333336 Student-2 Regdno = 301 Total marks = 164 Average marks = 54.666668

Passed all tests! ✔

➔Thus, program is executed successfully.